LOG2400 – Analyse et conception de logiciels

Hiver 2026

TP No. 4

Groupe 02

2335959 – Marc Agboidji

2396935 – David Pretty

2370407 – Félix Rolland

Soumis à :

Aziz Maarif

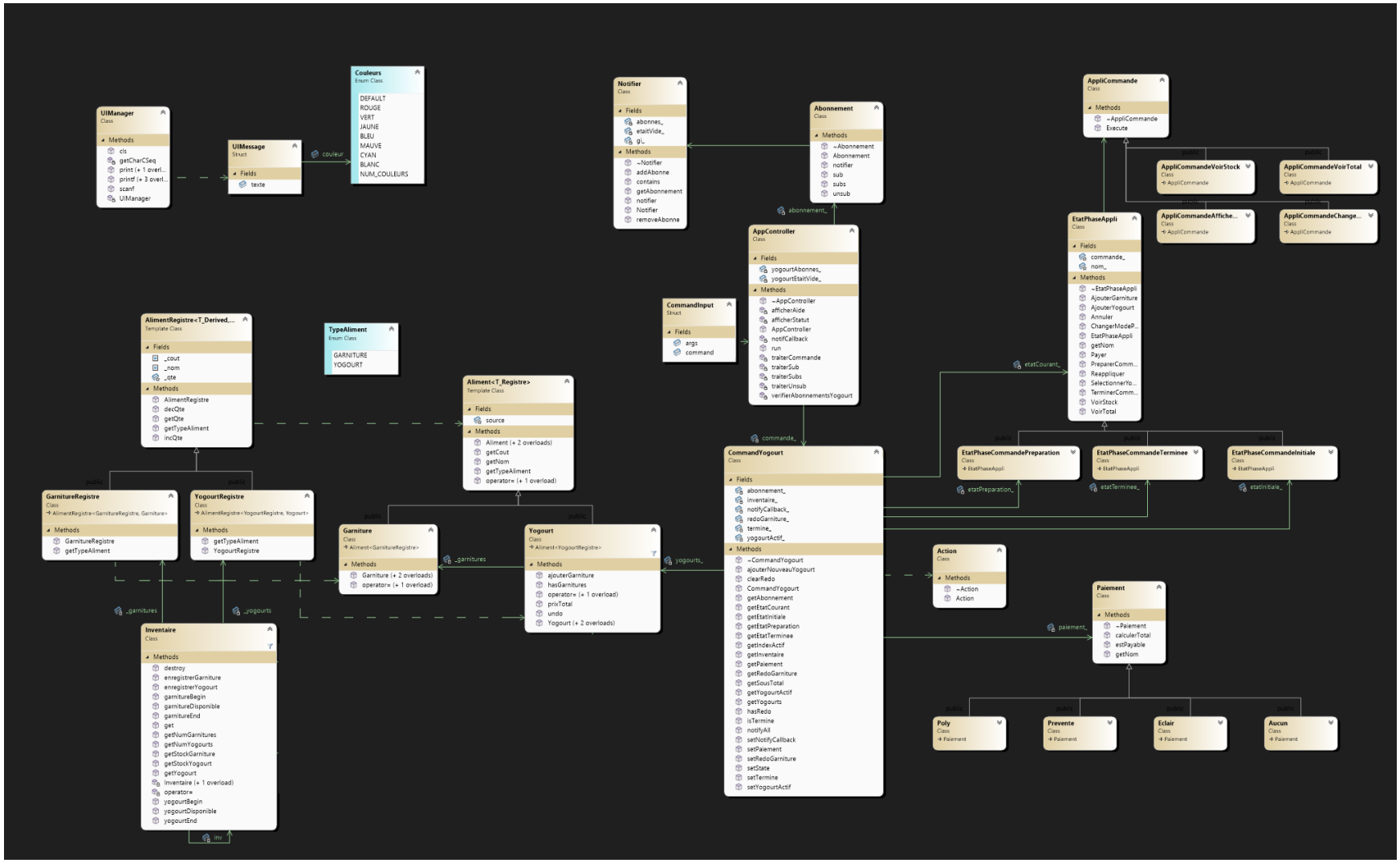
16 avril 2026

Table des matières

1. Présentation de notre conception 2
1.1 Diagramme de Classes 2
1.2 Présentation et justification des patrons choisis 3
1.3 Annexe : Ébauche de structure de classe 4

1. Présentation de notre conception

1.1 Diagramme de Classes



1.2 Présentation et justification des patrons choisis

Notre implémentation fait usage de beaucoup de patrons de conceptions, dont la plupart furent validé par notre chargé en séance via le pseudocode de déclaration de classes affiché à l'annexe :

Par exemple, dans le couple de classes `GarnitureRegistre` et `Garniture`, la classe `GarnitureRegistre` agit comme fabrique aux instances de `Garniture`, en étant la classe qui gère les instances de `Garniture` et leur création, dû à leurs constructeurs privés. Tant qu'à elle, la classe `Garniture`, étant une « instanciation » du concept d'une garniture, agit comme proxy aux propriétées définies par `GarnitureRegistre`. Nous avons conçu ces classes ainsi puisque ces patrons permettent d'établir un contrôle strict sur l'instanciation des garnitures, permettant ainsi que toute représentation de garniture sur un yogourt soient liés à une représentation interne en inventaire pour faciliter le contrôle de leurs attributs et leurs stocks. Les classes `YogourtRegistre` et `Yogourt` agissent similairement de par leur héritage des classes/gabarits `AlimentRegistre` et `Aliment`, à l'exception que `Yogourt` puisse se décorer par le patron décorateur de `Garnitures`.

La classe `Inventaire`, tant qu'à elle, fait usage du patron Singleton. Ce dernier est imposé par la suppression des constructeurs de copie, ainsi que la privatisation du constructeur générique. L'obtention de l'unique instance se fait par `Inventaire::get()`, une méthode statique qui gère un pointeur statique privé à l'instance d'`Inventaire`.

La gestion de la phase présente de l'application se fait par les classes `CommandYogourt` et `EtatPhaseAppli`, étant respectivement l'implémentation de la classe mère et de la classe état du patron état. Chaque état concret pour chaque phase de la commande (Initiale, préparation, terminée) hérite de la classe virtuelle `EtatPhaseAppli`. Dans celle-ci et ses dérivées, chaque méthode est implémentée via le patron commande via une classe dérivée la classe abstraite de base `AppliCommande`, afin de réduire la réutilisation de code entre chaque différent état dérivé de `EtatPhaseAppli`.

Pour la gestion de notifications, on utilise le patron observateur pour la simple raison qu'il s'agit d'une solution particulièrement optimisée pour ce genre de problème, ainsi que l'exemple typique d'un patron observateur est la gestion de notifications. Ici, les observateurs concrets sont la classe `Notifier`, et le diffuseur est la classe singleton `Abonnement`.

En somme, notre programme respecte majoritairement l'architecture MVC. Les seules exceptions sont que la vue est statique (voir `UIManager`), et que le contrôleur gère aussi la vue pour certaines situations.

1.3 Annexe : Ébauche de structure de classe

Ci-dessous se trouve l'ébauche de structure de programme / classes dont nous avons présentés et fait validé par notre chargé de TP en séance.

```
const static enum class Couleurs : unsigned char {
	DEFAULT,
	ROUGE,
	VERT,
	JAUNE,
	BLEU,
	MAUVE,
	CYAN,
	BLANC,
	NUM_COULEURS
};
struct UIMessage {
	std::string texte;
	Couleurs couleur = Couleurs::DEFAULT;
};
class UIManager {
public:
	static void print(const std::string_view& texte, Couleurs couleur =
Couleurs::DEFAULT);
	static void cls();
	static void print(const UIMessage& msg);
	static void printf(const std::string_view& texte, Couleurs couleur =
Couleurs::DEFAULT);
	static void printf(const UIMessage& msg);
	template<typename... Args>
	static void printf(const std::string_view& texte, Couleurs couleur =
Couleurs::DEFAULT, Args... args);
	template<typename... Args>
	static void printf(const UIMessage& msg, Args... args);
	static UIMessage& scanf();
private:
	UIManager() = delete;
	static constexpr const char* getCharCSeq(const Couleurs clr);
}
class EtatPhaseAppli {
	virtual AppliCommande& AjouterYogourt(string type);
	virtual AppliCommande& SelectionnerYogourt(uint index);
	virtual AppliCommande& AjouterGarniture(); // ouvrir menu
	virtual AppliCommande& Annuler();
	virtual AppliCommande& Reappliquer();
	virtual AppliCommande& TerminerCommande();
	virtual AppliCommande& PreparerCommande();
	virtual AppliCommande& ChangerModePaiment(string mode);
	virtual AppliCommande& Payer(); //todo, patron strategie
	virtual AppliCommande& VoirTotal();
	virtual AppliCommande& VoirStock();
}
class EtatPhaseCommandeIntitiale  : EtatPhaseAppli;
class EtatPhaseCommandePreparation         : EtatPhaseAppli;
class EtatPhaseCommandeTerminee            : EtatPhaseAppli;
class AppliCommande {
	virtual void Execute();
}
class AppliCommandeAfficherTexte : AppliCommande {
	AppliCommande(UIMessage& msg)
	void Execute() override;
private:
	UIMessage& msg
}
class Commande { // patron singleton
	static const uint MAX_YOGOURT = 2;
	uint calculerSousTotal();
	//etc...
	vector<CommandeYogourt>& yogourts;
}
class CommandeYogourt {
	CommandeYogourt(weak_ref<TypeYogourt>& type);
	void ajouterGarniture(weak_ref<Garniture> garniture);
	void undo();
	void redo();
	double calculerCoutCommande();
private:
	ActionHistory& _undo;
	vector<weak_ref<Garniture>>& garnitures;
	const weak_ref<TypeYogourt>& type;
};
//undo redo
class ActionHistory {
	void RegisterAction(Action&&);
	Action& UndoAction();
	Action& RedoAction();
	uint                        _indexcourant;
	vector<Action>&    _undohistory;
};
class Action {
	Action(Action::_type type,  weak_ref<Garniture>& garniture);
	const enum _type {
	AJOUTER,
	RETIRER,
	};
	const weak_ref<Garniture> _garniture;
}

//stuff d'inventaire
class Inventaire { // patron singleton, observateur
	uint     getNumGarnitures();
	weak_ref<Garniture>&        getGarniture(uint index);
	uint     getNumYogourts();
	weak_ref<Garniture>&        getYogourt(uint index);
private:
	const vector<unique_ptr<TypeYogourt>>& yogourts;
	const vector<unique_ptr<GarnitureRegistre>>& garnitures;
}
class GarnitureRegistre {
	Garniture(string nom, uint qte_initiale, double cout) :
	_cout(cout), _nom(nom)
	{};
	uint getQte();
	void incQte();
	Garniture decQte();
	const    double             _cout;
	const    string             _nom;
	friend class Garniture {
	double GetCout() const;
	string_view& getNom() const;
	private:
	Garniture();
	GarnitureRegistre& source;
	}
private:
	uint     _qte;
}
class TypeYogourt {
	TypeYogourt(string nom, double cout) :
	_cout(cout), _nom(nom)
	{};
	const    double             _cout;
	const    string             _nom;
}
```